

```
!pip install torch torchvision pillow
```

```
Requirement already satisfied: torch in /usr/local/lib/python3.12/dist-packages (2.9.0+cpu)
Requirement already satisfied: torchvision in /usr/local/lib/python3.12/dist-packages (0.24.0+cpu)
Requirement already satisfied: pillow in /usr/local/lib/python3.12/dist-packages (11.3.0)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from torch) (3.20.3)
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packages (from torch) (4.15.0)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch) (3.6.1)
Requirement already satisfied: Jinja2 in /usr/local/lib/python3.12/dist-packages (from torch) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-packages (from torch) (2025.3.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from torchvision) (2.0.2)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from Jinja2->torch) (3.0.3)
```

```
import os
print(os.listdir("/content"))
```

```
['.config', 'monet2photo', 'monet2photo.zip', 'sample_data']
```

```
print(os.listdir("/content/monet2photo"))
```

```
['testA', 'trainA', 'testB', 'trainB']
```

```
!pip install torch torchvision pillow tqdm
```

```
Requirement already satisfied: torch in /usr/local/lib/python3.12/dist-packages (2.9.0+cpu)
Requirement already satisfied: torchvision in /usr/local/lib/python3.12/dist-packages (0.24.0+cpu)
Requirement already satisfied: pillow in /usr/local/lib/python3.12/dist-packages (11.3.0)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (4.67.2)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from torch) (3.20.3)
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packages (from torch) (4.15.0)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch) (3.6.1)
Requirement already satisfied: Jinja2 in /usr/local/lib/python3.12/dist-packages (from torch) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-packages (from torch) (2025.3.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from torchvision) (2.0.2)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from Jinja2->torch) (3.0.3)
```

```
DATASET_PATH = "/content/monet2photo"
print(DATASET_PATH)
```

```
/content/monet2photo
```

```
import torch
print("GPU:", torch.cuda.is_available())
print("Device Name:", torch.cuda.get_device_name(0))
```

```
GPU: True
Device Name: Tesla T4
```

```
!wget https://efrosgans.eecs.berkeley.edu/cyclegan/datasets/monet2photo.zip
!unzip monet2photo.zip
```

```

inflating: monet2photo/testA/00090.jpg
inflating: monet2photo/testA/00710.jpg
inflating: monet2photo/testA/00240.jpg
inflating: monet2photo/testA/00650.jpg
inflating: monet2photo/testA/00330.jpg
inflating: monet2photo/testA/01130.jpg
inflating: monet2photo/testA/00860.jpg
inflating: monet2photo/testA/01200.jpg
inflating: monet2photo/testA/00220.jpg
inflating: monet2photo/testA/01100.jpg
inflating: monet2photo/testA/00260.jpg
inflating: monet2photo/testA/00070.jpg
inflating: monet2photo/testA/00730.jpg
inflating: monet2photo/testA/00640.jpg
inflating: monet2photo/testA/00520.jpg
inflating: monet2photo/testA/01300.jpg
inflating: monet2photo/testA/00950.jpg
inflating: monet2photo/testA/00140.jpg
inflating: monet2photo/testA/01250.jpg
inflating: monet2photo/testA/00855.jpg
inflating: monet2photo/testA/01270.jpg
inflating: monet2photo/testA/01280.jpg
inflating: monet2photo/testA/00590.jpg
inflating: monet2photo/testA/00550.jpg
inflating: monet2photo/testA/00970.jpg
inflating: monet2photo/testA/00290.jpg
inflating: monet2photo/testA/00040.jpg
inflating: monet2photo/testA/00910.jpg
inflating: monet2photo/testA/00610.jpg
inflating: monet2photo/testA/00150.jpg
inflating: monet2photo/testA/00320.jpg
inflating: monet2photo/testA/00270.jpg
inflating: monet2photo/testA/01140.jpg
inflating: monet2photo/testA/00670.jpg
inflating: monet2photo/testA/00830.jpg
inflating: monet2photo/testA/00050.jpg
inflating: monet2photo/testA/00960.jpg
inflating: monet2photo/testA/00430.jpg
inflating: monet2photo/testA/00060.jpg
inflating: monet2photo/testA/00210.jpg
inflating: monet2photo/testA/00190.jpg
inflating: monet2photo/testA/00200.jpg
inflating: monet2photo/testA/00460.jpg

```

```

import os
print(os.listdir("/content/monet2photo"))

```

```
['testB', 'trainA', 'trainB', 'testA']
```

```

import torch

DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
print("Using Device:", DEVICE)

```

```
Using Device: cuda
```

```

import os
import itertools
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
from torchvision import transforms
from PIL import Image

# -----
# CONFIG
# -----
DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
EPOCHS = 20
BATCH_SIZE = 1
LR = 0.0002
IMG_SIZE = 128
DATASET_PATH = "/content/monet2photo"

print("Using Device:", DEVICE)

# -----
# DATASET (LIMITED FAST MODE)

```

```

# -----
class ImageDataset(Dataset):
    def __init__(self, root, mode="train", max_samples=50):
        self.root = root
        self.mode = mode
        self.max_samples = max_samples

        self.transform = transforms.Compose([
            transforms.Resize((IMG_SIZE, IMG_SIZE)),
            transforms.ToTensor(),
            transforms.Normalize((0.5,0.5,0.5),(0.5,0.5,0.5))
        ])

        self.files_A = sorted(os.listdir(f"{root}/{mode}A"))[:max_samples]
        self.files_B = sorted(os.listdir(f"{root}/{mode}B"))[:max_samples]

    def __getitem__(self, index):
        img_A = Image.open(f"{self.root}/{self.mode}A/{self.files_A[index % len(self.files_A)]}").convert("RGB")
        img_B = Image.open(f"{self.root}/{self.mode}B/{self.files_B[index % len(self.files_B)]}").convert("RGB")
        return self.transform(img_A), self.transform(img_B)

    def __len__(self):
        return max(len(self.files_A), len(self.files_B))

# -----
# MODELS
# -----
class ResnetBlock(nn.Module):
    def __init__(self, dim):
        super().__init__()
        self.block = nn.Sequential(
            nn.ReflectionPad2d(1),
            nn.Conv2d(dim, dim, 3),
            nn.InstanceNorm2d(dim),
            nn.ReLU(True),
            nn.ReflectionPad2d(1),
            nn.Conv2d(dim, dim, 3),
            nn.InstanceNorm2d(dim)
        )

    def forward(self, x):
        return x + self.block(x)

class Generator(nn.Module):
    def __init__(self):
        super().__init__()

        layers = [
            nn.ReflectionPad2d(3),
            nn.Conv2d(3,64,7),
            nn.InstanceNorm2d(64),
            nn.ReLU(True)
        ]

        dim = 64
        for _ in range(2):
            layers += [
                nn.Conv2d(dim, dim*2,3,2,1),
                nn.InstanceNorm2d(dim*2),
                nn.ReLU(True)
            ]
            dim *= 2

        for _ in range(6): # reduced blocks = faster training
            layers += [ResnetBlock(dim)]

        for _ in range(2):
            layers += [
                nn.ConvTranspose2d(dim, dim//2,3,2,1,output_padding=1),
                nn.InstanceNorm2d(dim//2),
                nn.ReLU(True)
            ]
            dim //= 2

        layers += [
            nn.ReflectionPad2d(3),
            nn.Conv2d(64,3,7),

```

```

        nn.Tanh()
    ]

    self.model = nn.Sequential(*layers)

    def forward(self,x):
        return self.model(x)

class Discriminator(nn.Module):
    def __init__(self):
        super().__init__()
        self.model = nn.Sequential(
            nn.Conv2d(3,64,4,2,1),
            nn.LeakyReLU(0.2,True),

            nn.Conv2d(64,128,4,2,1),
            nn.InstanceNorm2d(128),
            nn.LeakyReLU(0.2,True),

            nn.Conv2d(128,256,4,2,1),
            nn.InstanceNorm2d(256),
            nn.LeakyReLU(0.2,True),

            nn.Conv2d(256,512,4,1,1),
            nn.InstanceNorm2d(512),
            nn.LeakyReLU(0.2,True),

            nn.Conv2d(512,1,4,1,1)
        )

    def forward(self,x):
        return self.model(x)

# -----
# INIT MODELS
# -----
G_AB = Generator().to(DEVICE)
G_BA = Generator().to(DEVICE)
D_A = Discriminator().to(DEVICE)
D_B = Discriminator().to(DEVICE)

criterion_GAN = nn.MSELoss()
criterion_cycle = nn.L1Loss()

optimizer_G = optim.Adam(
    itertools.chain(G_AB.parameters(), G_BA.parameters()),
    lr=LR, betas=(0.5,0.999)
)

optimizer_D_A = optim.Adam(D_A.parameters(), lr=LR, betas=(0.5,0.999))
optimizer_D_B = optim.Adam(D_B.parameters(), lr=LR, betas=(0.5,0.999))

# -----
# DATALOADER
# -----
dataset = ImageDataset(DATASET_PATH, max_samples=50)
loader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True)

# -----
# TRAINING LOOP
# -----
for epoch in range(EPOCHS):
    for i,(real_A, real_B) in enumerate(loader):

        real_A, real_B = real_A.to(DEVICE), real_B.to(DEVICE)

        # Fix: Create valid and fake tensors with shape matching Discriminator output
        valid = torch.ones_like(D_A(real_A)).to(DEVICE)
        fake = torch.zeros_like(valid)

        # ---- Train Generators ----
        optimizer_G.zero_grad()

        fake_B = G_AB(real_A)
        fake_A = G_BA(real_B)

        loss_GAN = criterion_GAN(D_B(fake_B), valid) + criterion_GAN(D_A(fake_A), valid)

```

```

recov_A = G_BA(fake_B)
recov_B = G_AB(fake_A)

loss_cycle = criterion_cycle(recov_A, real_A) + criterion_cycle(recov_B, real_B)

loss_G = loss_GAN + 10 * loss_cycle
loss_G.backward()
optimizer_G.step()

# ---- Train D_A ----
optimizer_D_A.zero_grad()
loss_D_A = (criterion_GAN(D_A(real_A), valid) +
            criterion_GAN(D_A(fake_A.detach()), fake)) * 0.5
loss_D_A.backward()
optimizer_D_A.step()

# ---- Train D_B ----
optimizer_D_B.zero_grad()
loss_D_B = (criterion_GAN(D_B(real_B), valid) +
            criterion_GAN(D_B(fake_B.detach()), fake)) * 0.5
loss_D_B.backward()
optimizer_D_B.step()

print(f"Epoch [{epoch+1}/{EPOCHS}] Loss_G: {loss_G.item():.4f}")

print("Training Finished ✅")

```

```

Using Device: cuda
Epoch [1/20] Loss_G: 8.2495
Epoch [2/20] Loss_G: 6.4616
Epoch [3/20] Loss_G: 7.5119
Epoch [4/20] Loss_G: 5.3649
Epoch [5/20] Loss_G: 6.6024
Epoch [6/20] Loss_G: 5.2693
Epoch [7/20] Loss_G: 5.7650
Epoch [8/20] Loss_G: 5.2155
Epoch [9/20] Loss_G: 5.3604
Epoch [10/20] Loss_G: 4.7336
Epoch [11/20] Loss_G: 5.6614
Epoch [12/20] Loss_G: 5.6341
Epoch [13/20] Loss_G: 5.7071
Epoch [14/20] Loss_G: 4.8826
Epoch [15/20] Loss_G: 7.1083
Epoch [16/20] Loss_G: 8.8108
Epoch [17/20] Loss_G: 4.3491
Epoch [18/20] Loss_G: 5.0889
Epoch [19/20] Loss_G: 11.6205
Epoch [20/20] Loss_G: 4.9077
Training Finished ✅

```

```

import matplotlib.pyplot as plt
import torchvision.transforms as transforms
from PIL import Image
import torch
import os

# -----
# PATH
# -----
test_path = "/content/monet2photo/testA"

# -----
# TRANSFORM (Same as Training)
# -----
transform = transforms.Compose([
    transforms.Resize((128,128)),
    transforms.ToTensor(),
    transforms.Normalize((0.5,0.5,0.5),(0.5,0.5,0.5))
])

# -----
# DENORMALIZE FUNCTION
# -----
def denormalize(t):
    return t * 0.5 + 0.5

# -----

```

```

# LOAD TEST IMAGES
# -----
test_images = sorted(os.listdir(test_path))

# -----
# GENERATE + SHOW 5 IMAGES
# -----
G_AB.eval()

plt.figure(figsize=(15,12))

for i in range(5):

    img_name = test_images[i]

    # Load Image
    img = Image.open(f"{test_path}/{img_name}").convert("RGB")
    inp = transform(img).unsqueeze(0).to(DEVICE)

    # Generate Output
    with torch.no_grad():
        out = G_AB(inp)

    # Convert for display
    inp_show = denormalize(inp[0]).cpu()
    out_show = denormalize(out[0]).cpu()

    # Plot Input
    plt.subplot(5,2,2*i+1)
    plt.imshow(inp_show.permute(1,2,0))
    plt.title("Monet Input")
    plt.axis("off")

    # Plot Output
    plt.subplot(5,2,2*i+2)
    plt.imshow(out_show.permute(1,2,0))
    plt.title("Generated Photo")
    plt.axis("off")

plt.tight_layout()
plt.show()

```

Monet Input



Generated Photo



Monet Input



Generated Photo



Monet Input



Generated Photo



Start coding or [generate](#) with AI.



Monet Input



Monet Input

Generated Photo



Generated Photo